

Dimensional Data Modeling & Analytical Warehouse Architecture

Target Audience: Engineers transitioning from writing transactional SQL queries to architecting enterprise-grade Analytical Data Warehouses (PostgreSQL, DuckDB, Snowflake, BigQuery, Databricks).

Table of Contents

- 1. The Paradigm Shift: OLTP (3NF) vs OLAP (Dimensional)
- 2. The Core Primitive: Table Grain
- 3. Facts vs Dimensions: The Anatomy of Analytics
- 4. The Kimball 4-Step Dimensional Design Process
- 5. Star Schema vs Snowflake Schema: Deep Architectural Trade-Offs
- 6. Keys in Data Warehousing: Surrogate vs Natural vs Business Keys
- 7. Specialized Dimensions
 - 7.1 Role-Playing Dimensions
 - 7.2 Degenerate Dimensions
 - 7.3 Conformed Dimensions
 - 7.4 The "Unknown / Missing" Member (-1 Record Pattern)
- 8. Fact Table Archetypes & Measurement Additivity
 - 8.1 Transaction Fact Tables
 - 8.2 Periodic Snapshot Fact Tables
 - 8.3 Accumulating Snapshot Fact Tables
 - 8.4 Factless Fact Tables
- 9. Capstone Implementation: Operational (OLTP) to Star Schema (OLAP)
- 10. Master Dimensional Modeling Cheat Sheet & Anti-Patterns

1. The Paradigm Shift: OLTP vs OLAP

In Weeks 1 and 2, you wrote SQL to manipulate data inside existing tables. Data modeling asks a higher-order engineering question: **How should the tables themselves be structured so that thousands of analytical questions can be answered fast, accurately, and without complex joins?**

OLTP (Online Transaction)
Goal: Maximize write throughput
Normalization: 3NF (Third Normal Form)
Pattern: Hundreds of narrow tables
Updates: Heavy row-level CRUD
Joins: 10-20 joins to get full context
Optimization: Primary Key, B-Trees

OLAP (Online Analytics)
Goal: Maximize read/scan throughput
Normalization: Denormalized / Star
Pattern: Central Facts + Wide Dims
Updates: Bulk append / Partition drop
Joins: 1-3 star joins directly to fact
Optimization: Columnar, Partitioning

Why 3NF Fails for Analytics

Operational databases (PostgreSQL/MySQL backing web apps) use **3NF (Third Normal Form)** to eliminate data redundancy. In 3NF:

- Every non-key attribute must depend on "the key, the whole key, and nothing but the key."
- Customer address is split into `addresses`, `cities`, `states`, `countries`, `postal_codes`.
- Product is split into `products`, `brands`, `categories`, `subcategories`.

If an analyst or BI tool runs a query to calculate `Monthly Sales by Product Category and Customer State`:

- In 3NF, the query planner must execute an **8-to-12 table relational join**.
- Table joins produce exponential intermediate Cartesian products.
- BI tools generate horrendous SQL that crushes operational database CPU and memory.

Dimensional modeling deliberately denormalizes these relationships into wide dimensions surrounding numeric fact tables.

2. The Core Primitive: Table Grain

Golden Rule of Data Warehousing:

"Never write a single line of DDL, ETL, or SQL before you can state the grain of the table in a single, unambiguous sentence."

The **Grain** is the fundamental atomic level of detail represented by a single row in a table. It is the physical contract of your dataset.

Examples of Table Grain

Table	What 1 Row Represents (The Grain)	Anti-Pattern Violation
<code>fact_order_line</code>	One specific product line item within a customer order	Grouping by <code>order_id</code> and mixing total order shipping fee on every item line (causes duplicate counting).
<code>fact_daily_inventory</code>	One product at one warehouse at midnight on one calendar date	Storing multiple status changes per day without a timestamp or snapshot marker.
<code>dim_customer</code>	One unique legal customer entity as of the current state	Storing multiple address history rows without an SCD Type 2 surrogate key and version flag.

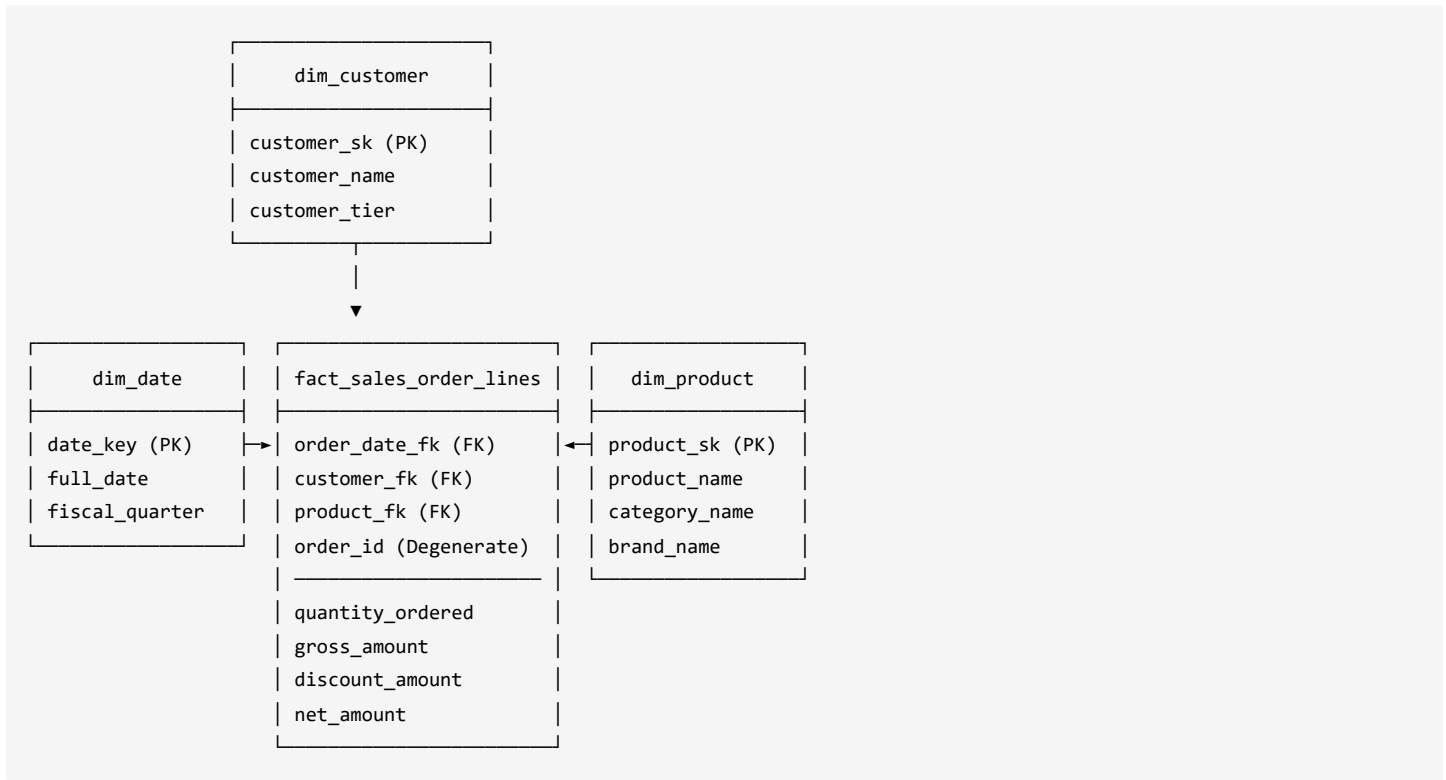
How to Detect a Broken Grain

If you ever find yourself needing `DISTINCT` or `GROUP BY` inside a basic join to avoid duplicate rows, **your grain is violated or mismatched**.

```
-- SMELL: If you need to do this, your join created a Fan-Out because grains didn't match!
SELECT COUNT(DISTINCT o.order_id), SUM(f.line_amount)
FROM fact_orders o
JOIN fact_order_items f ON o.order_id = f.order_id;
```

3. Facts vs Dimensions

A dimensional warehouse splits all reality into two concepts: **Measurements (Facts)** and **Context (Dimensions)**.



1. Facts (Numeric Measurements)

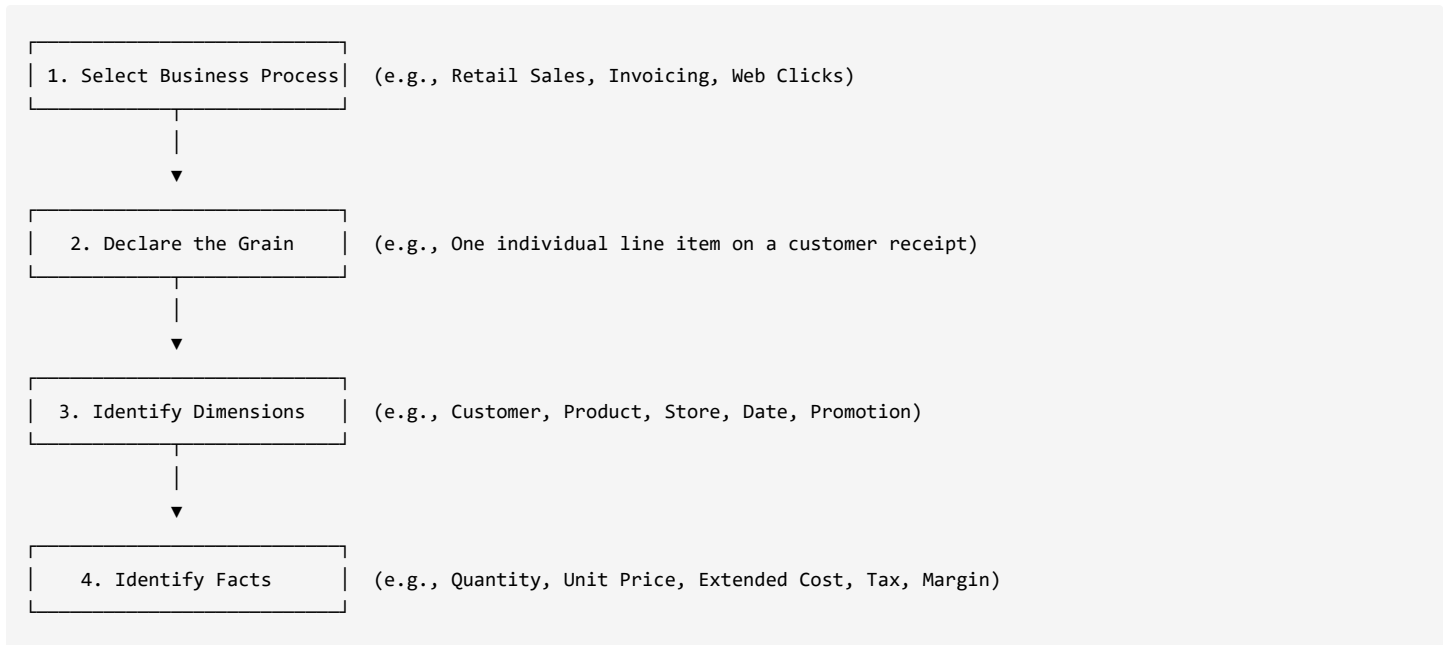
- **What they are:** Quantitative numeric data generated by a business event (a sale, a page click, an inventory check, a sensor reading).
- **Characteristics:**
 - Highly normalized, append-heavy, billions of rows.
 - Mostly integer foreign keys and decimal numbers.
 - Thin and tall (few columns, massive row counts).

2. Dimensions (Descriptive Context)

- **What they are:** The "who, what, where, when, why, and how" surrounding a business event.
- **Characteristics:**
 - Rich, qualitative, textual attributes (`customer_segment` , `region` , `product_category`).
 - Used for `WHERE` filters, `GROUP BY` slices, and report labels.
 - Wide and short (many columns, smaller row counts compared to facts).

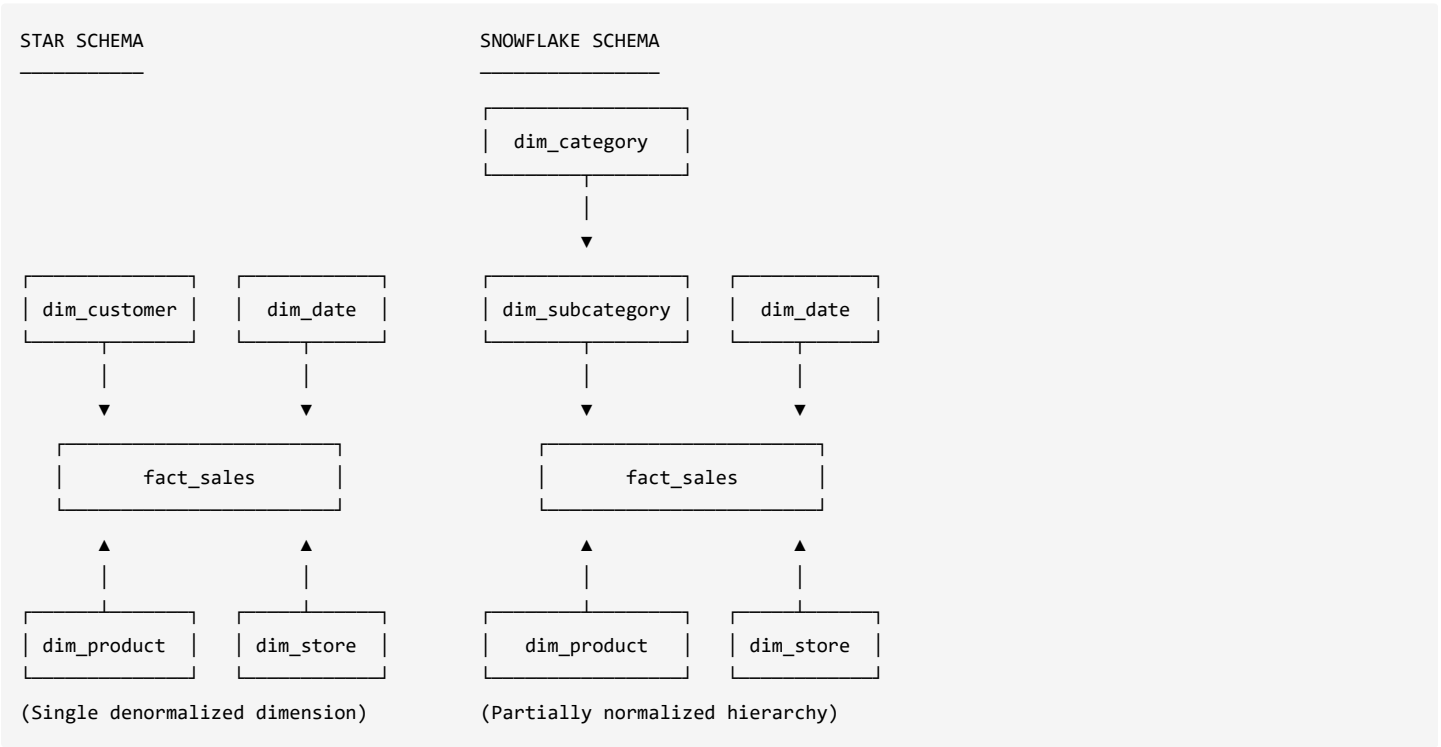
4. The Kimball 4-Step Dimensional Design Process

Formulated by Ralph Kimball, every dimensional model should be designed using these sequential 4 steps:



- 1. **Select the Business Process:** Pick a discrete operational activity, not a department. Department-based modeling creates silos; business process modeling creates enterprise bus architecture.
- 2. **Declare the Grain:** Be as atomic as possible. Never aggregate at the grain stage (e.g. do not choose "Daily sales by store" if you have line-item receipts). Atomic grain gives maximum downstream slicing flexibility.
- 3. **Identify Dimensions:** Ask: *What describes the event at this exact grain?*
- 4. **Identify Facts:** Ask: *What numeric metrics occur at this exact grain?*

5. Star Schema vs Snowflake Schema



Comprehensive Comparison Matrix

Architectural Dimension	Star Schema	Snowflake Schema
Normalization	Denormalized (3NF ignored in dimensions)	Partially normalized (dimensions split into sub-tables)
Query Complexity	Very Low (Single-level joins directly from Fact to Dim)	Moderate to High (Requires multi-hop joins across hierarchies)
Query Performance	Fastest (Optimizers use Star Join bitmaps; columnar formats compress repeated strings efficiently)	Slower (More joins, larger planner overhead)
Storage Footprint	Slightly higher in legacy row-stores; negligible in modern columnar stores (Parquet/ORC/Snowflake)	Lower storage due to deduplicated string dimensions
Maintenance & ETL	Slightly more ETL work to flatten hierarchies into one row	Easier updates if an attribute name changes (single row update)
BI Tool Usability	Ideal (Self-service analysts easily navigate 1 fact and direct dimensions)	Confusing (Analysts struggle to know which sub-table to join)

Modern Data Engineering Consensus:
Use **Star Schema** by default. With modern columnar compression (dictionary encoding, Run-Length Encoding) and cheap cloud storage, the minor storage savings of Snowflake schema do not justify the query complexity and join penalties.

6. Keys in Data Warehousing

1. Natural Key (Business Key)

- The identifier generated by the source operational application (e.g. `customer_uuid` , `order_number` , `employee_code`).
- Problem in Warehousing:** Natural keys change over time, can contain invalid formats, get recycled across multiple acquired source systems, or are slow composite strings.

2. Surrogate Key (Warehouse Key)

- An artificial, sequentially generated integer or bigint managed exclusively by the Data Warehouse (`customer_sk INT GENERATED ALWAYS AS IDENTITY`).
- Why Surrogate Keys are Mandatory:**
 - i. **Performance:** Integer joins (`INT` / `BIGINT`) are 5x–10x faster and use far less RAM than joining 36-character UUID strings or compound keys.
 - ii. **System Independence:** Isolates the warehouse from source operational migrations (e.g., Salesforce customer IDs vs homegrown PostgreSQL customer IDs).
 - iii. **History Tracking (SCD Type 2):** A single customer natural key (`CUST-1001`) can have 5 rows in `dim_customer` representing address changes. Each row has a unique `customer_sk` .
 - iv. **The -1 Unknown Row:** Allows fact tables to link to surrogate key `-1` when a dimension reference is missing or null.

7. Specialized Dimensions

7.1 Role-Playing Dimensions

A single physical dimension table that is referenced multiple times in the same fact table under different operational roles.

- **Classic Example:** `dim_date`
 - In `fact_sales`, you have:
 - `order_date_fk`
 - `shipping_date_fk`
 - `delivery_date_fk`
 - `payment_date_fk`
- **Implementation:** You do NOT create four separate date tables. You create **one** physical `dim_date` table and create SQL views or alias the table in queries:

```
SELECT
    f.order_id,
    d_order.calendar_date AS order_date,
    d_ship.calendar_date AS shipping_date,
    d_deliv.calendar_date AS delivery_date
FROM fact_sales f
JOIN dim_date d_order ON f.order_date_fk = d_order.date_key
JOIN dim_date d_ship ON f.shipping_date_fk = d_ship.date_key
JOIN dim_date d_deliv ON f.delivery_date_fk = d_deliv.date_key;
```

7.2 Degenerate Dimensions

A dimension key or attribute that is stored directly in the fact table without joining to a separate dimension table.

- **Classic Examples:** `order_number`, `invoice_id`, `bill_of_lading_number`, `transaction_reference`.
- **Why:** These identifiers have the exact same 1-to-1 cardinality as the fact row itself. Creating a `dim_order_number` table would result in a useless table with 100 million rows containing only an ID.
- **Usage:** Kept in the fact table for transaction drill-through, auditability, and grouping line items.

7.3 Conformed Dimensions

A dimension that has the exact same meaning, business definition, and key structure across multiple different business processes and fact tables.

- Example: `dim_customer` and `dim_date` are shared between `fact_sales`, `fact_support_tickets`, and `fact_subscription_billing`.
- **The Kimball Bus Architecture:** When dimensions are conformed, you can perform **Drill-Across queries** across multiple fact tables without creating spaghetti joins.

7.4 The "Unknown / Missing" Member (-1 Pattern)

In production, operational data is dirty:

- An order arrives with `customer_id = NULL` or references a deleted customer ID.
- If your fact table has an `INNER JOIN` or `FOREIGN KEY` constraint, the pipeline fails or drops revenue data.
- **The Solution:** Seed every dimension with a row with surrogate key `-1`:

```
INSERT INTO dim_customer (customer_sk, natural_customer_id, customer_name, country)
VALUES (-1, 'UNKNOWN', 'Unknown / Not Provided', 'N/A');
```

- In your Fact ETL:

```
COALESCE(dim.customer_sk, -1) AS customer_fk
```

- **Result:** Outer joins are avoided, referential integrity is preserved, and queries never drop fact records!

8. Fact Table Archetypes & Additivity

Measure Additivity Types

1. **Fully Additive:** Can be meaningfully summed across *any* dimension (e.g. `revenue` , `quantity_sold` , `cost`).
2. **Semi-Additive:** Can be summed across some dimensions (e.g., products, stores), but **cannot** be summed across `dim_date` (e.g. `account_balance` , `warehouse_inventory_count`). Summing your bank balance across 30 days gives a nonsensical number; you must take the ending balance or average.
3. **Non-Additive:** Cannot be summed across any dimension (e.g. `unit_price` , `discount_percentage` , `profit_margin_ratio`). Ratios must be recalculated from additive sums: `SUM(profit) / SUM(revenue)` .

The 3 Core Fact Table Archetypes

1. Transaction Fact	2. Periodic Snapshot Fact	3. Accumulating Snapshot Fact
Grain: 1 event occurrence	Grain: 1 snapshot period (Day)	Grain: 1 entire lifecycle entity
Rows added: Continuous stream	Rows added: Once per period	Rows added: 1 per order, updated
Example: Individual cash swipe	Example: Account balance at EOD	Example: Order fulfillment stages
Dates: Single transaction date	Dates: Snapshot date	Dates: Multi-date milestones

9. Capstone Implementation (Mapped to Your Live PostgreSQL Database)

In this week's code artifacts, we transform your existing 3NF database (`catalog` , `customer` , `orders` , `logistics`) containing 10,000 orders and ~30,000 order items into a clean `dwh` Star Schema:

1. `01_oltp_operational_schema.sql` : Audits your live 3NF database schemas, benchmarks relational join bottlenecks, and sets up `dwh` .
2. `02_date_dimension_generator.sql` : Generates `dwh.dim_date` (2024–2030) with `YYYYMMDD` keys, fiscal quarters, and `-1` unknown member.
3. `03_conformed_dimensions.sql` : Transforms `customer.customers` + `customer.addresses` into `dwh.dim_customer` , `catalog.products` + `catalog.categories` into `dwh.dim_product` , and `logistics.warehouses` into `dwh.dim_warehouse` .
4. `04_role_playing_dimensions.sql` : Demonstrates multi-date role playing across order, shipping, and delivery dates.
5. `05_degenerate_dimensions.sql` : Analyzes transaction numbers (`order_number` , tracking codes) stored directly on facts.
6. `06_fact_table_grain_and_ddl.sql` : Establishes `dwh.fact_sales_order_lines` with explicit grain contract and additive measures.
7. `07_fact_etl_pipeline.sql` : The production ELT pipeline loading all 29,983 order items with surrogate key replacement and tax/shipping allocation.

8. `08_star_vs_snowflake.sql` : Benchmarks Star vs Snowflake schema on your actual 100 products and 20 categories with `EXPLAIN ANALYZE`.
 9. `09_accumulating_snapshot_fact.sql` : Builds `dwh.fact_order_fulfillment_accumulating` tracking fulfillment pipeline turnaround on your 8,000 shipments.
 10. `10_analytical_star_queries.sql` : High-performance BI queries on your real data contrasting 3-join Star queries vs 7-join 3NF queries.
-

10. Master Dimensional Modeling Cheat Sheet

Dimensional Design Checklist

- ☐ Have you written down the grain in plain English?
- ☐ Are all dimensions using integer Surrogate Keys instead of natural UUIDs?
- ☐ Does every dimension table contain a `-1` row for missing/late-arriving data?
- ☐ Are your fact measures strictly numeric? (Never put strings in fact tables; use degenerate dimensions or dim foreign keys).
- ☐ Are ratios pre-calculated in the fact table? (Anti-pattern! Never store pre-computed averages or percentages. Store the numerator and denominator as additive metrics).
- ☐ Are dates modeled via `dim_date` integer surrogate keys (`YYYYMMDD`)?
- ☐ Is the schema resilient to source changes?

The 5 Deadly Sins of Data Modeling

1. **The Centipede Fact Table:** A fact table with 35 foreign keys joining to tiny 1-column tables (over-normalization).
2. **Mixing Grains in One Fact Table:** Storing order header shipping costs on the line-item fact row without allocating or using a separate header fact table.
3. **Snowflaking for the Sake of Purity:** Breaking `dim_product` into 5 sub-tables to save 4 MB of disk space, costing 10x slower queries.
4. **Natural Keys as Foreign Keys:** Joining facts to dimensions on 36-character `VARCHAR` UUIDs or customer email addresses.
5. **Pre-aggregating Percentages:** Storing `margin_pct = 0.25` in fact rows and then running `AVG(margin_pct)` in BI dashboards (statistically invalid mathematical fallacy).